Monitoreo con APM y prueba de estrés sobre una aplicación CRUD dockerizada: determinación del límite de la infraestructura

Carlos Mario Ayala Ceballos y Santiago Gonzales Becerra

Programa de Ingeniería de Software, Fundación Escuela Tecnológica de Neiva "Jesús Oviedo Pérez"

Optativa IV – Sistemas de Tiempo Real Distribuidos

Ing. Juan Carlos Polania Cortes

19 de octubre de 2026

Resumen

Este informe documenta la instrumentación de una aplicación CRUD de productos (Node.js, Express y PostgreSQL) con un sistema de monitoreo del rendimiento de aplicaciones (APM) basado en OpenTelemetry, Grafana, Tempo y Prometheus, y su sometimiento a una prueba de estrés con k6, todo ejecutado en contenedores Docker. La carga se aplicó en siete escalones de tres minutos, entre 100 y 2000 usuarios concurrentes, con una mezcla de 70 % lecturas, 20 % creaciones y 10 % actualizaciones, y se evaluó contra tres criterios de saturación declarados antes de medir: tasa de error de hasta 1 %, latencia p95 de hasta 500 ms y crecimiento sostenido del throughput. La infraestructura sostuvo 900 usuarios concurrentes con aproximadamente 895 peticiones por segundo, p95 de 12 ms y 0 % de error, usando 10 conexiones simultáneas a la base de datos. Por encima de ese punto el throughput se estancó cerca de 900 peticiones por segundo y la latencia p95 superó los 500 ms. La evidencia del APM y de las estadísticas de los contenedores muestra que el recurso limitante es el procesador del contenedor de la aplicación, que alcanzó entre 98 % y 99 % de su límite de una vCPU, mientras la base de datos no superó el 71 % de su núcleo. Un experimento que triplicó el tamaño del conjunto de conexiones no elevó el throughput, lo que confirma que la espera por conexiones observada en las trazas es un síntoma de la saturación del procesador y no su causa.

Palabras clave: APM, OpenTelemetry, prueba de estrés, k6, punto de saturación, Docker

Monitoreo con APM y prueba de estrés sobre una aplicación CRUD dockerizada: determinación del límite de la infraestructura

Este trabajo continúa los laboratorios anteriores de la asignatura: se parte de una aplicación CRUD con base de datos y se le agregan un sistema de monitoreo de rendimiento de aplicaciones (APM) y una prueba de carga que la lleve hasta su límite (Polania Cortes, 2026). El propósito no es demostrar que el sistema aguanta mucho, sino poder afirmar con datos propios dónde está el límite de la infraestructura y qué recurso lo impone.

La pregunta que guía el informe es la planteada en la guía del taller: ¿cuántas peticiones por segundo y cuántas conexiones concurrentes soporta esta infraestructura antes de incumplir los criterios declarados, y qué recurso impone ese límite? Para responderla se siguió el enfoque de utilización, saturación y errores propuesto por Gregg (2020): para cada recurso (procesador, memoria, conexiones) se midió cuánto se usa, cuánto trabajo queda esperando y si aparecen errores.

Como regla del taller, los cuatro componentes (aplicación, base de datos, APM y generador de carga) se ejecutan dentro de contenedores Docker, y el equipo anfitrión solo requiere Docker, Postman, el navegador y un editor de código.

Método

Arquitectura del entorno

No se encontró en el equipo el proyecto de los laboratorios 1 y 3, por lo que se construyó una aplicación CRUD de productos que cumple las condiciones del punto de partida: expone las operaciones de crear, listar, consultar, actualizar y eliminar, y su base de datos corre en un contenedor con un volumen persistente. La Tabla 1 resume los componentes y los límites de

recursos asignados a cada contenedor. Todo el entorno se define en un único archivo de composición (Docker, s. f.) y se levanta con el comando `docker compose up -d --build`.

Tabla 1

Componentes del entorno y límites de recursos asignados

Componente	Tecnología	Contenedor	CPU	Memoria
Aplicación CRUD	Node.js 20, Express 4, pg (pool de 10)	app	1,0	512 MiB
Base de datos	PostgreSQL 16 (max_connections = 100)	db	1,0	1 GiB
APM	Grafana, Tempo, Prometheus y colector OTel	apm	4,0	4 GiB
Generador de carga	k6 0.54	k6	4,0	2 GiB

Nota. El APM corresponde a la imagen grafana/otel-lgtm, que reúne en un contenedor la opción "Grafana + Tempo + Prometheus" sugerida por la guía (Grafana Labs, s. f.-a). Los límites se fijaron para que los resultados no dependieran de la actividad del equipo anfitrión.

Instrumentación

La aplicación se instrumentó con el SDK de OpenTelemetry para Node.js y su autoinstrumentación de los módulos http, express y pg (OpenTelemetry Authors, s. f.). Cada petición genera una traza con el span HTTP de entrada, los spans del middleware de Express, un span `pg-pool.connect` que mide la espera por una conexión libre del conjunto de conexiones y un span `pg.query` que mide la consulta SQL. Esto permite separar el tiempo consumido dentro de la aplicación del tiempo consumido en la base de datos, que es lo que justifica usar un APM y no solo las métricas del generador de carga.

Las métricas se exportan cada 5 s por OTLP/HTTP. La duración de las peticiones se registra como histograma con cubetas finas entre 1 ms y 10 s, de modo que los percentiles p50, p95 y p99 calculados en Prometheus sean precisos, y se etiqueta por método, ruta y código de estado, lo que da peticiones por segundo, latencia y tasa de error por operación. Además se publican cuatro métricas propias del conjunto de conexiones (abiertas, libres, en espera y

máximo), tomadas del cliente node-postgres (Carlson, s. f.). El muestreo de trazas se dejó en 100 % y el APM estuvo encendido durante la línea base, para que su costo quedara incluido en la medición.

Diseño de la prueba de estrés

El escenario se escribió en un archivo de k6 versionado (Grafana Labs, s. f.-b). Cada usuario virtual ejecuta en ciclo una operación elegida al azar con la siguiente proporción: 50 % listar una página aleatoria de 20 productos, 20 % consultar un producto por identificador aleatorio entre 10 000 sembrados, 20 % crear y 10 % actualizar un producto aleatorio. En total son 70 % lecturas, 20 % creaciones y 10 % actualizaciones. El nombre, la descripción, el precio y las existencias cambian en cada petición, para no medir solo el efecto de una caché. Entre peticiones cada usuario espera entre 0,5 y 1,5 s, de modo que cada usuario genera cerca de una petición por segundo.

La carga se aplicó en escalones crecientes: un calentamiento de un minuto con 10 usuarios, descartado del análisis, seguido de siete escalones de tres minutos con 100, 300, 600, 900, 1200, 1500 y 2000 usuarios concurrentes. De cada escalón se excluyeron los primeros 15 s y los últimos 5 s para no mezclar la transición entre escalones. La progresión sugerida por la guía (10 a 400 usuarios) se ajustó después de una corrida de sondeo, que no forma parte de los resultados, en la que 800 usuarios solo ocupaban el 68 % del procesador de la aplicación; con la progresión original no se habría llegado a la saturación.

Durante la prueba, un script muestreó cada 3 s las estadísticas de los contenedores (`docker stats`) y el número de sesiones activas en PostgreSQL (`pg_stat_activity`). Al terminar, un contenedor de análisis consultó Prometheus y Tempo en la ventana de cada escalón, calculó la tabla de resultados, evaluó los criterios y generó las gráficas.

Criterios de saturación

Los tres criterios de la Tabla 2 se declararon por escrito el 19 de octubre de 2026, antes de ejecutar la primera medición, y no se modificaron después. Un escalón cumple solo si satisface los tres a la vez, y el punto de saturación es el último escalón que los cumple.

Tabla 2

Criterios de saturación declarados antes de medir

Criterio	Umbral	Significado y fuente
Tasa de error	≤ 1 %	Respuestas 5xx en el APM más fallos de red o tiempos de espera de 10 s registrados por k6
Latencia p95	≤ 500 ms	Percentil 95 del lado del servidor, histograma http.server.duration del APM
Crecimiento del throughput	≥ 90 % del esperado	Esperado = throughput del escalón anterior × (usuarios actuales / usuarios anteriores); por debajo se considera aplanado

Nota. Declaración completa en docs/CRITERIOS_SATURACION.md del repositorio del laboratorio.

Equipo de medición

La medición se realizó en el equipo descrito en la Tabla 3. Durante la prueba solo estaban activos los contenedores del laboratorio y un contenedor ajeno (open-webui) en reposo, con un consumo cercano al 0,2 % de un núcleo.

Tabla 3

Ficha del equipo donde se realizó la medición

Elemento	Valor
Procesador	Intel Core i7-13700HX (13.ª generación)
Núcleos e hilos	16 núcleos (8 de rendimiento y 8 de eficiencia), 24 hilos
Memoria RAM	40 GB
Sistema operativo	Windows 11 Pro for Workstations, compilación 10.0.26300
Docker	Docker Desktop, motor 29.6.1, Compose v5.1.4, backend WSL 2
Recursos de la máquina virtual de Docker	24 CPU y 19,4 GiB de memoria
Límites por contenedor	Ver Tabla 1

Resultados

Línea base en reposo

Con la aplicación, la base de datos y el APM encendidos pero sin carga, durante 60 s, la aplicación consumió 0,1 % de un núcleo y 35 MiB de memoria, la base de datos 0,4 % y 48 MiB, y el APM 1,7 %, sin sesiones activas en PostgreSQL. Ese consumo es despreciable frente al de la prueba, así que los valores de las tablas siguientes se deben casi por completo a la carga.

Rendimiento por escalón

La Tabla 4 presenta el throughput, la latencia y la tasa de error de cada escalón, tomados del APM, y la Tabla 5 el consumo de recursos y el estado de las conexiones a la base de datos. Hasta 900 usuarios el throughput creció en proporción exacta a los usuarios y la latencia se mantuvo en pocos milisegundos. Desde 1200 usuarios el throughput dejó de crecer y la latencia aumentó en dos órdenes de magnitud.

Tabla 4

Throughput, latencia y tasa de error por escalón (corrida oficial)

Esc.	Usuarios	Pet./s	p50 (ms)	p95 (ms)	p99 (ms)	Error (%)
1	100	100,1	1,5	2,8	3,8	0,00
2	300	299,5	1,5	2,9	6,2	0,00
3	600	597,7	1,6	4,1	8,8	0,00
4	**900**	**894,6**	**1,9**	**12,1**	**27,9**	**0,00**
5	1200	896,5	298,6	575,0	735,7	0,00
6	1500	917,8	625,0	857,6	988,9	0,00
7	2000	865,2	1269,1	1686,2	1937,2	0,30

Nota. Valores calculados sobre la ventana de cada escalón en Prometheus. La tasa de error es el mayor valor entre las respuestas 5xx del APM y los fallos registrados por k6. La fila en negrita es el punto de saturación.

Tabla 5

Consumo de recursos y conexiones a la base de datos por escalón

Esc.	Usuarios	CPU app	CPU BD	Memoria app	Sesiones PG	Pool abiertas	Pool en espera
1	100	11 %	6 %	48 MiB	4	3,6 / 10	0,0 (máx. 0)
2	300	30 %	17 %	52 MiB	7	6,6 / 10	0,0 (máx. 0)
3	600	55 %	33 %	58 MiB	10	9,8 / 10	0,0 (máx. 0)
4	**900**	**78 %**	**49 %**	**63 MiB**	**10**	**10,0 / 10**	**0,4 (máx. 12)**
5	1200	98 %	59 %	76 MiB	10	10,0 / 10	297 (máx. 440)
6	1500	98 %	61 %	92 MiB	10	10,0 / 10	589 (máx. 755)
7	2000	99 %	60 %	115 MiB	10	10,0 / 10	1114 (máx. 1276)

Nota. CPU en porcentaje de un núcleo; el límite de la aplicación y de la base de datos es 100 % (una vCPU). Sesiones PG: sesiones en pg_stat_activity, frente a un máximo configurado de 100. Pool abiertas: conexiones abiertas del conjunto de la aplicación frente a su tamaño máximo. Pool en espera: peticiones esperando una conexión libre, promedio y máximo.

La Tabla 6 aplica los criterios declarados a cada escalón. Los cuatro primeros cumplen los tres criterios. El escalón de 1200 usuarios incumple dos: su p95 (575 ms) supera el umbral y su throughput es solo el 75 % del esperado. Por lo tanto, el punto de saturación es el escalón 4, con 900 usuarios concurrentes.

Tabla 6

Evaluación de los criterios de saturación por escalón

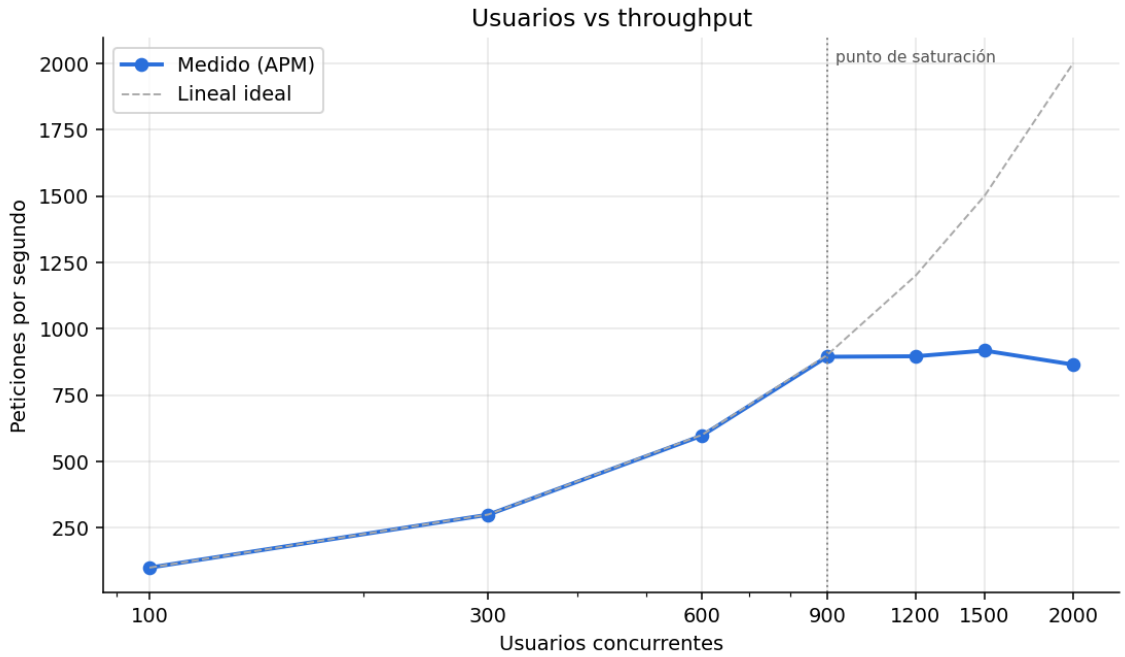
Esc.	Usuarios	Error ≤ 1 %	p95 ≤ 500 ms	Throughput ≥ 90 %	¿Cumple?
1	100	Sí (0,00 %)	Sí (3 ms)	No aplica	Sí
2	300	Sí (0,00 %)	Sí (3 ms)	Sí (100 %)	Sí
3	600	Sí (0,00 %)	Sí (4 ms)	Sí (100 %)	Sí
4	**900**	**Sí (0,00 %)**	**Sí (12 ms)**	**Sí (100 %)**	**Sí**
5	1200	Sí (0,00 %)	No (575 ms)	No (75 %)	No
6	1500	Sí (0,00 %)	No (858 ms)	No (82 %)	No
7	2000	Sí (0,30 %)	No (1686 ms)	No (71 %)	No

Nota. El criterio de throughput no se aplica al primer escalón porque no tiene un escalón anterior con el cual compararse.

Las Figuras 1, 2 y 3 son las tres gráficas exigidas por la guía. Las Figuras 4, 5 y 6 aportan la evidencia sobre el recurso que satura.

Figura 1

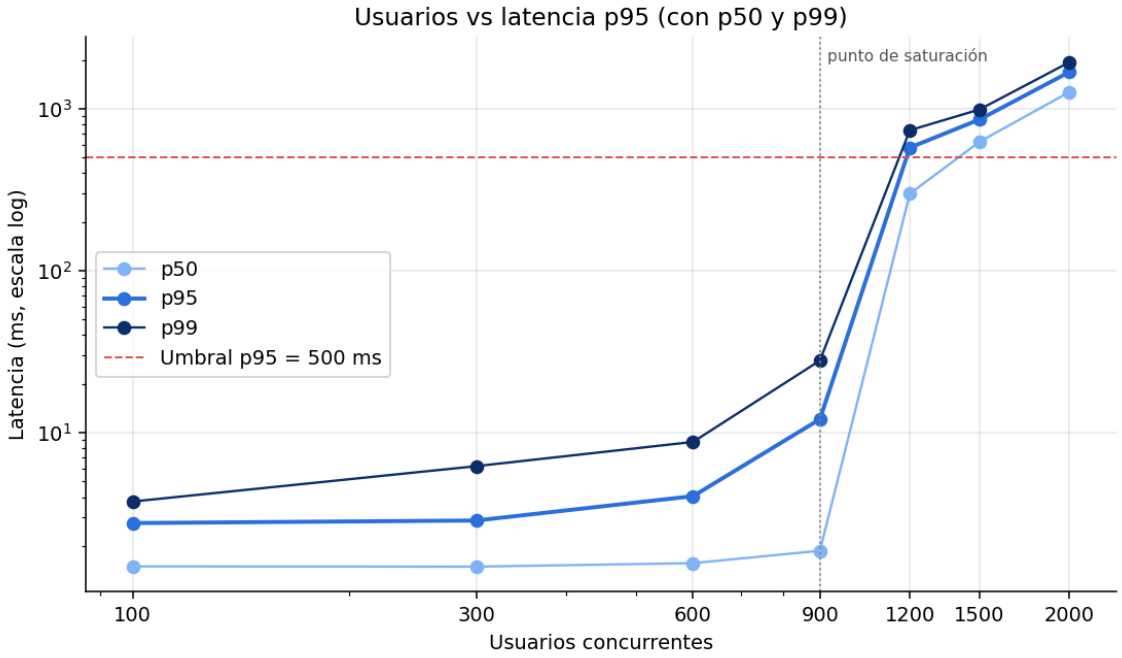
Usuarios concurrentes frente a peticiones por segundo



Nota. La línea discontinua es el crecimiento lineal ideal (una petición por segundo por usuario). La línea vertical punteada marca el punto de saturación (900 usuarios). Eje horizontal en escala logarítmica.

Figura 2

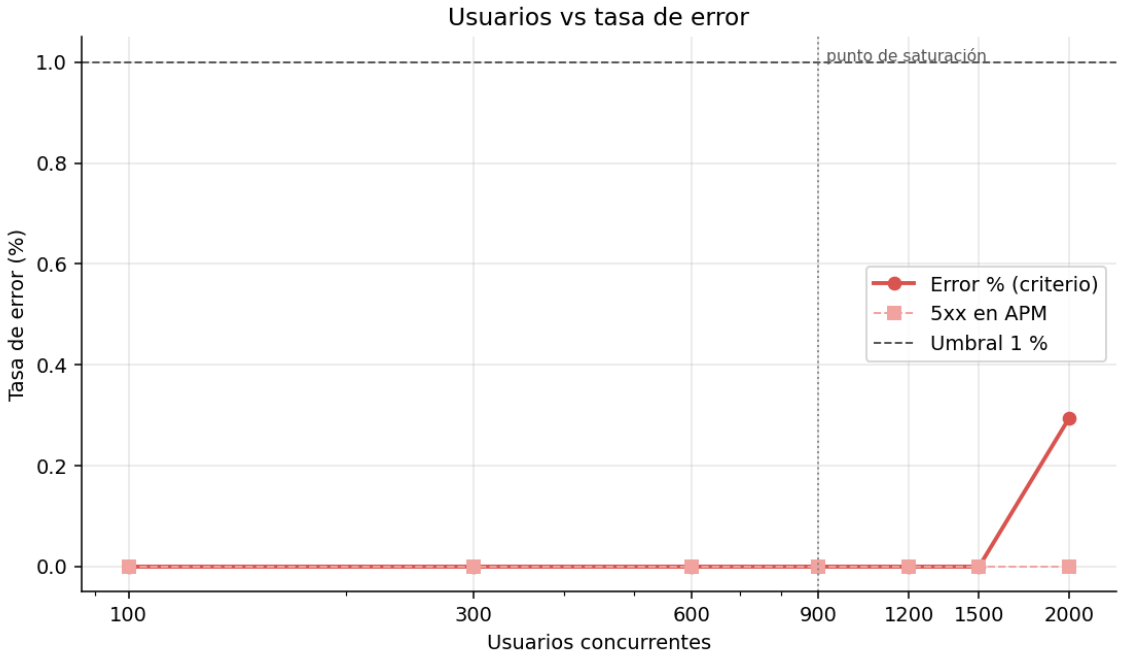
Usuarios concurrentes frente a latencia p95, con p50 y p99 como referencia



Nota. Eje vertical en escala logarítmica. La línea roja discontinua es el umbral declarado de 500 ms para el p95.

Figura 3

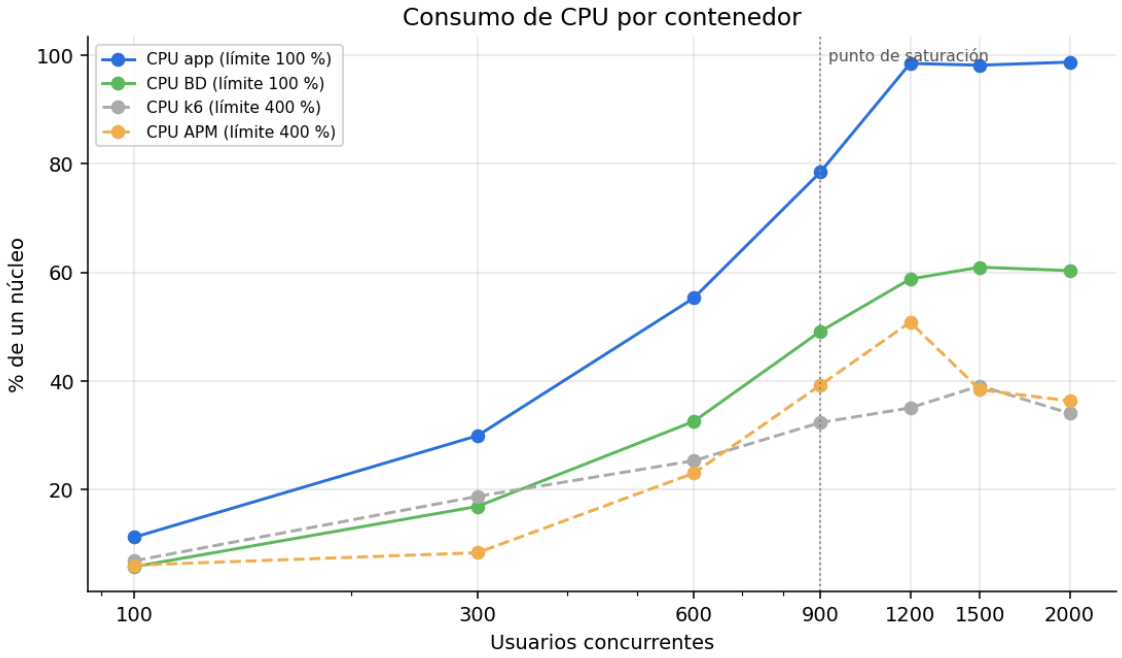
Usuarios concurrentes frente a tasa de error



Nota. La línea continua es la tasa usada en el criterio (máximo entre APM y k6); la discontinua son las respuestas 5xx registradas por el APM, que se mantuvieron en cero.

Figura 4

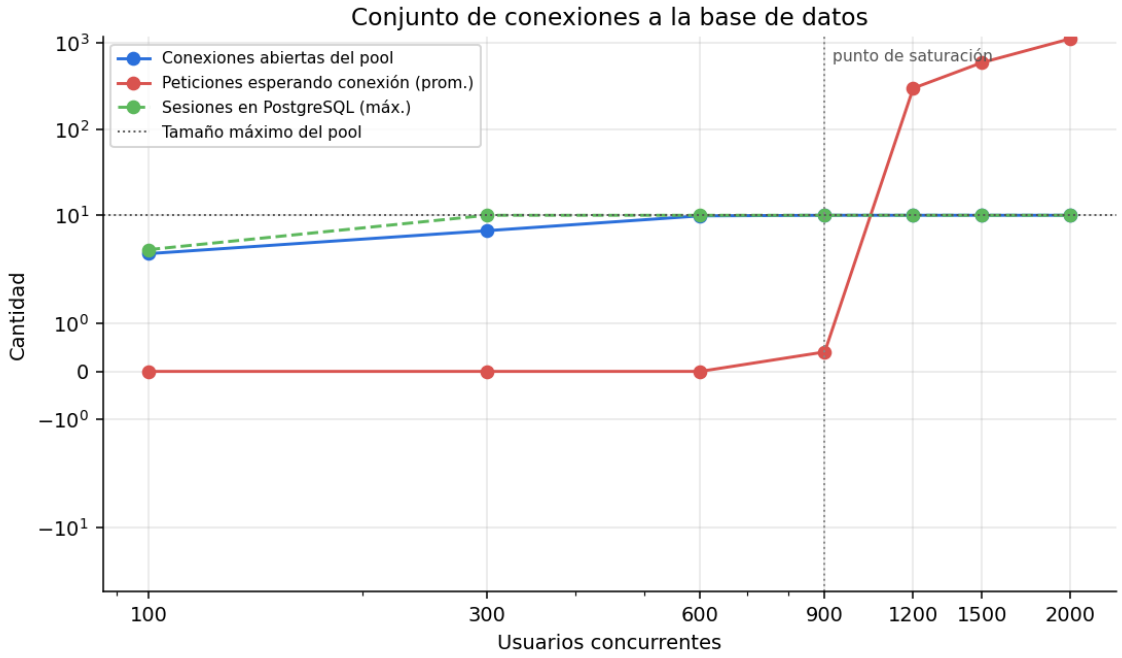
Consumo de procesador por contenedor



Nota. Porcentaje de un núcleo. La aplicación y la base de datos tienen un límite de 100 %; k6 y el APM, de 400 %.

Figura 5

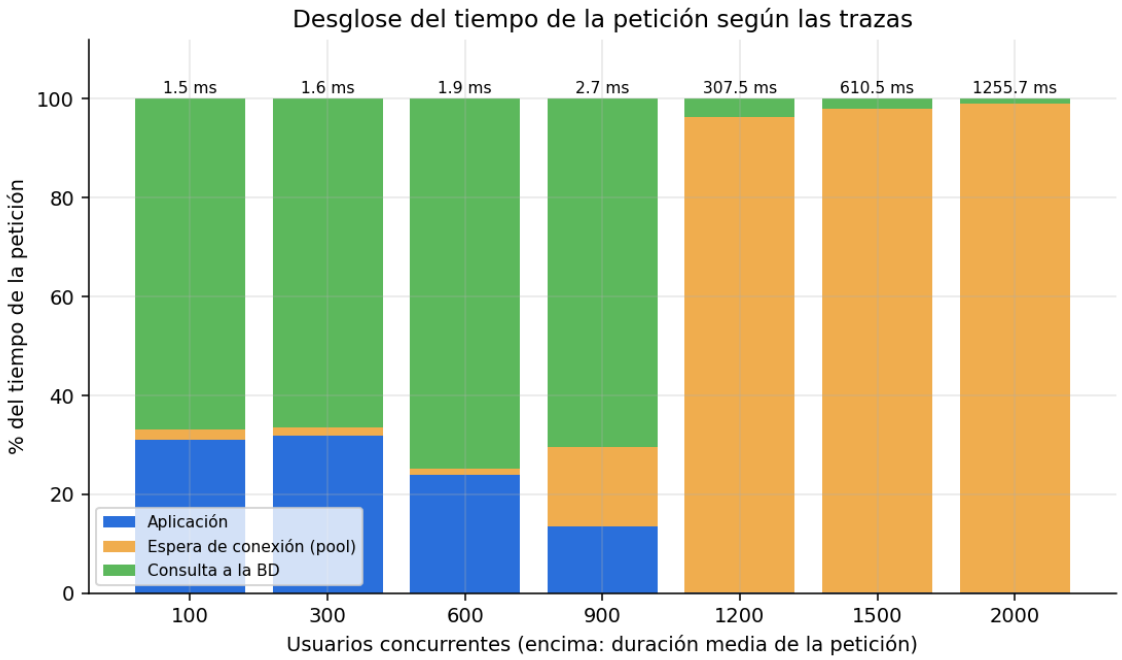
Estado del conjunto de conexiones a la base de datos



Nota. Eje vertical en escala logarítmica simétrica. El conjunto de conexiones tiene un máximo de 10.

Figura 6

Distribución del tiempo de una petición según las trazas del APM



Nota. Muestra de 150 trazas por escalón tomadas de Tempo. “Aplicación” es el tiempo de la petición que no corresponde a spans de PostgreSQL. Sobre cada barra se indica la duración media de la petición.

Verificación del generador de carga

Para descartar que el generador de carga fuera el cuello de botella, se registró también su consumo. El procesador de k6 promedió entre 7 % y 39 % de un límite de 400 % (máximo puntual de 118 %), y el del APM entre 6 % y 51 % del mismo límite. Las cifras de k6 coinciden con las del APM: en el escalón de 900 usuarios k6 completó 161 084 peticiones (cerca de 895 por segundo) con un p95 de 25,7 ms, que incluye el tiempo de red. Por lo tanto, la prueba midió el sistema y no el generador.

Confirmación del punto de saturación

Siguiendo la recomendación de la guía, se repitió el tramo de 900 a 1200 usuarios (tres minutos cada uno). Además, para distinguir entre dos causas posibles del límite, se hizo una tercera corrida con el conjunto de conexiones ampliado de 10 a 30. La Tabla 7 compara las tres corridas.

Tabla 7

Repetición del escalón de saturación y experimento con un conjunto de conexiones mayor

Corrida	Usuarios	Pet./s	p95 (ms)	CPU app	CPU BD	Pool (abiertas / espera)	¿Cumple?
Oficial, pool 10	900	894,6	12,1	78 %	49 %	10 / 0,4	Sí
Oficial, pool 10	1200	896,5	575,0	98 %	59 %	10 / 297	No
Repetición, pool 10	900	892,3	26,7	80 %	65 %	10 / 5,9	Sí
Repetición, pool 10	1200	923,9	489,7	98 %	70 %	10 / 240	No
Pool 30	900	893,0	14,0	83 %	61 %	30 / 0,0	Sí
Pool 30	1200	895,2	497,2	99 %	71 %	30 / 270	No

Nota. En la repetición y en el experimento, el escalón de 1200 usuarios cumple el p95 por un margen estrecho, pero incumple el criterio de throughput (78 % y 75 % del esperado, respectivamente).

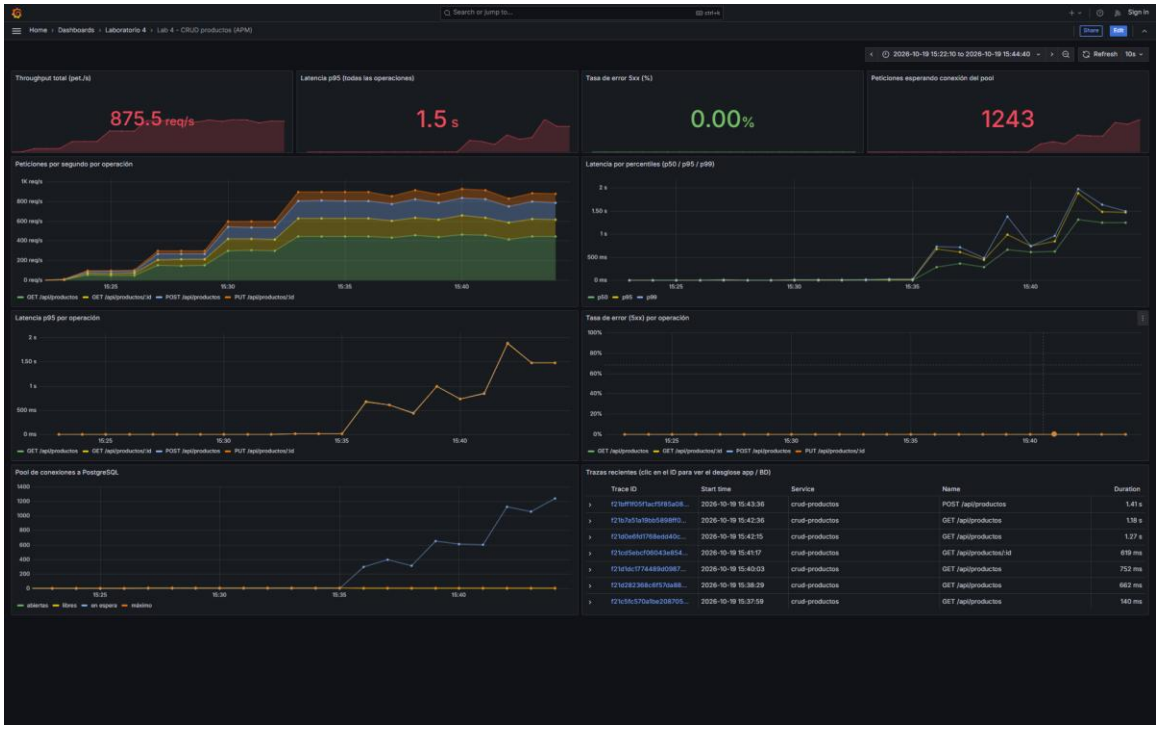
El resultado se sostiene: en las tres corridas, 900 usuarios cumple los criterios, 1200 no los cumple, y el techo de throughput queda entre 890 y 925 peticiones por segundo. Con 30 conexiones el throughput fue el mismo (895 peticiones por segundo), el procesador de la aplicación siguió en 99 % y siguió habiendo unas 270 peticiones esperando, aunque había tres veces más conexiones disponibles.

Capturas del APM

Las Figuras 7 a 10 muestran el APM durante la corrida oficial, entre las 15:22 y las 15:45 del 19 de octubre de 2026, en el panel aprovisionado “Lab 4 - CRUD productos (APM)” de Grafana.

Figura 7

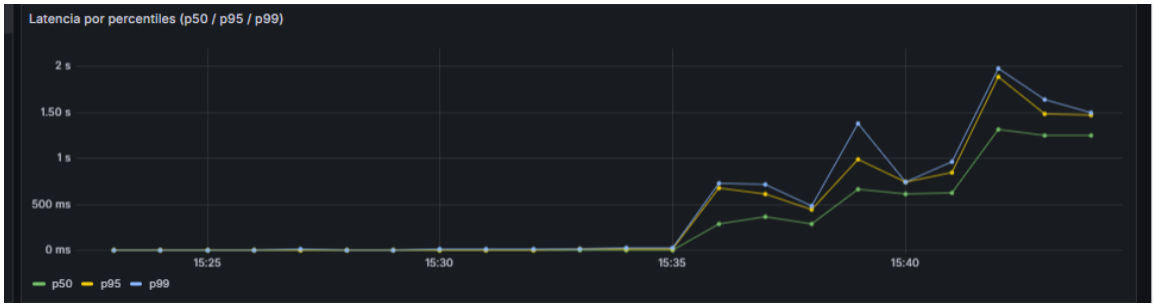
Panel de servicio del APM durante la corrida oficial



Nota. Captura del panel “Lab 4 - CRUD productos (APM)” en Grafana, rango de 15:22:10 a 15:44:40. Los indicadores superiores muestran el último valor del rango, que corresponde al escalón de 2000 usuarios. Las peticiones por segundo suben por escalones hasta cerca de 900 (hacia las 15:31) y luego se aplanan, mientras la latencia y las peticiones en espera del conjunto de conexiones crecen desde las 15:34, cuando comienza el escalón de 1200 usuarios.

Figura 8

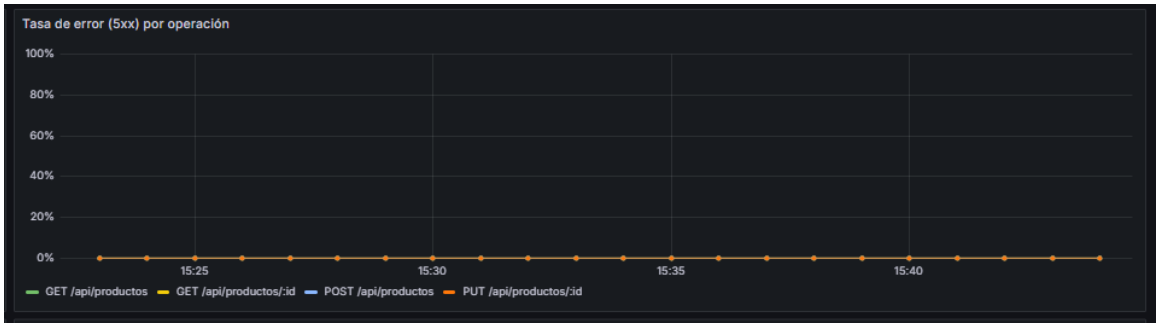
Latencia por percentiles (p50, p95 y p99) en el APM



Nota. Panel “Latencia por percentiles”. Los tres percentiles se mantienen en pocos milisegundos hasta las 15:34 y saltan a cientos de milisegundos al pasar de 900 a 1200 usuarios; cada escalón posterior los eleva más.

Figura 9

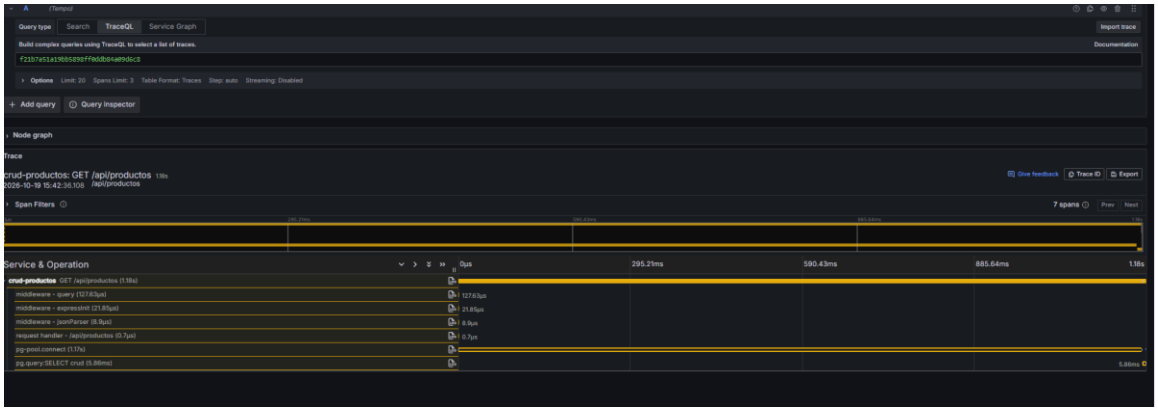
Tasa de error por operación en el APM



Nota. Panel “Tasa de error (5xx) por operación”. Ninguna operación registró respuestas 5xx durante la prueba; los 462 fallos del escalón de 2000 usuarios fueron registrados solo por k6, porque el servidor no llegó a responder esas peticiones.

Figura 10

Traza individual desglosada en Tempo



Nota. Traza de una petición GET /api/productos a las 15:42:36, durante el escalón de 2000 usuarios. De su duración total de 1,18 s, el span pg-pool.connect (espera de una conexión libre) ocupa 1,17 s, la consulta pg.query solo 5,86 ms y el middleware de Express menos de 0,2 ms.

Discusión

Respuesta a la pregunta central

La infraestructura medida sostiene **900 usuarios concurrentes con 894,6 peticiones por segundo** sin incumplir ninguno de los criterios declarados (p95 de 12,1 ms y 0 % de error), usando 10 conexiones simultáneas a PostgreSQL. Al pasar a 1200 usuarios el throughput se estanca en torno a 900 peticiones por segundo (entre 865 y 924 en todas las corridas) y la latencia p95 sube a entre 490 y 575 ms. **El recurso que impone el límite es el procesador del contenedor de la aplicación**: llegó a entre 98 % y 99 % de su límite de una vCPU justo cuando el throughput se aplanó, mientras la base de datos se quedó entre 59 % y 71 % de su núcleo, la memoria de la aplicación usó como máximo 115 MiB de 512 MiB y el generador de carga estuvo lejos de su límite.

Preguntas de análisis

¿Qué se degradó primero al cruzar el punto de saturación: la latencia o la tasa de error?

Se degradó primero la latencia. Entre 900 y 1200 usuarios el p95 pasó de 12 ms a 575 ms, unas 47 veces más, y el p50 de 1,9 ms a 299 ms, mientras la tasa de error se mantuvo en 0,00 %. El error solo apareció en el último escalón (2000 usuarios), con 0,30 %: 462 peticiones fallidas en k6 y ninguna respuesta 5xx en el APM. Es decir, fueron peticiones que el cliente dio por perdidas sin que el servidor llegara a responder con un error.

Ese orden indica que el sistema no rechaza trabajo cuando se satura, sino que lo encola. Las peticiones que exceden la capacidad esperan en la cola del bucle de eventos de Node.js y en la del conjunto de conexiones, que llegó a tener 1276 peticiones esperando. Como no hay control de admisión, la saturación aparece primero como tiempo de espera, y los errores solo llegarían cuando esa espera supere los tiempos límite (5 s para obtener una conexión y 10 s en el cliente). Las cifras cumplen la ley de Little (Little, 1961): con 896 peticiones por segundo y una duración media de 0,31 s, el número esperado de peticiones en curso es de unas 275, del mismo orden que las 297 en espera medidas en el escalón de 1200 usuarios.

¿Qué proporción del tiempo de una petición se consume en la aplicación y en la base de datos, y cambió con la carga?

Según las trazas, sin saturación unos dos tercios del tiempo de una petición corresponden a la consulta a la base de datos y un tercio a la aplicación (Tabla 8). La proporción cambió por completo al saturar. Desde 1200 usuarios, entre el 96 % y el 99 % del tiempo es espera por una conexión del conjunto, mientras el trabajo real casi no crece (unos 0,8 ms de aplicación y unos 11 ms de consulta). La petición no tarda más porque la consulta sea más lenta, sino porque

espera su turno. La Figura 10 lo muestra en una traza concreta: de 1,18 s, la espera por conexión ocupa 1,17 s y la consulta apenas 5,86 ms.

Tabla 8

Distribución del tiempo de una petición según las trazas del APM

Usuarios	Duración media	Aplicación	Espera de conexión	Consulta a la BD
100	1,5 ms	31 %	2 %	67 %
300	1,6 ms	32 %	2 %	66 %
600	1,9 ms	24 %	1 %	75 %
900	2,7 ms	13 %	16 %	70 %
1200	307,5 ms	0 % (0,8 ms)	96 %	4 % (11,5 ms)
1500	610,5 ms	0 % (0,7 ms)	98 %	2 % (12,6 ms)
2000	1255,7 ms	0 % (0,9 ms)	99 %	1 % (10,7 ms)

Nota. Muestra de 150 trazas por escalón de la corrida oficial, tomadas de Tempo.

Que la espera por conexión domine no significa que el conjunto de conexiones sea el cuello de botella. Con el procesador de la aplicación al 99 %, el bucle de eventos tarda en procesar las respuestas de PostgreSQL y por eso las conexiones se liberan tarde. El experimento de la Tabla 7 lo confirma: con 30 conexiones la espera siguió siendo de unas 270 peticiones y el throughput no cambió.

¿El throughput siguió creciendo con cada escalón o se aplanó?

Creció de forma lineal, con 100 % de la eficiencia esperada, hasta 900 usuarios, y luego se aplanó en unas 900 peticiones por segundo: 894,6; 896,5; 917,8 y 865,2 peticiones por segundo con 900, 1200, 1500 y 2000 usuarios (Figura 1). Con 2000 usuarios incluso bajó un poco, por el costo de manejar más peticiones en espera. Las corridas de confirmación dieron 923,9 y 895,2 peticiones por segundo con 1200 usuarios, así que el techo está entre 865 y 925 peticiones por segundo.

¿Qué se agotó primero: el procesador, la memoria o el conjunto de conexiones?

Se agotó primero el procesador de la aplicación. El conjunto de conexiones aparece lleno antes (9,8 de 10 desde 600 usuarios), pero eso no limitó el sistema: con 600 y 900 usuarios el throughput siguió creciendo de forma lineal y casi no había peticiones esperando (entre 0 y 0,4 en promedio). Además, con un conjunto de 30 conexiones el techo fue el mismo. En cambio, el procesador de la aplicación creció en proporción a la carga (11 %, 30 %, 55 % y 78 %) y llegó al 98–99 % justo cuando el throughput se aplanó (Figura 4).

La memoria nunca fue un problema: el máximo fue 115 MiB de 512 MiB, un 22 %. Tampoco lo fue la base de datos: su procesador se quedó entre 59 % y 71 % y el máximo de sesiones (31, en el experimento con 30 conexiones) estuvo lejos de su límite de 100. El costo medido es de cerca de 1,1 ms de procesador de la aplicación por petición (98 % de un núcleo dividido entre unas 900 peticiones por segundo). Ese costo incluye la instrumentación con muestreo al 100 % y el hecho de que Node.js ejecuta el código JavaScript en un único hilo (OpenJS Foundation, s. f.).

Cambio propuesto para elevar el límite

Se propone escalar horizontalmente la aplicación: dos réplicas de una vCPU, o el módulo cluster de Node.js con dos procesos y dos vCPU, detrás de un balanceador de carga, con 10 conexiones por réplica. Las mediciones respaldan este cambio por cuatro razones:

1. El recurso que satura es el procesador de un único proceso de Node.js, que solo puede usar un núcleo. Repartir la carga entre dos procesos duplica el procesador disponible para la aplicación.
2. El siguiente cuello de botella sería la base de datos. Con unas 895 peticiones por segundo, PostgreSQL usó entre 49 % y 71 % de su núcleo (entre 0,55 y 0,8 ms de

procesador por petición), lo que da un techo de entre 1250 y 1800 peticiones por segundo. Por eso se espera pasar de unas 900 a entre 1250 y 1500 peticiones por segundo, no al doble.

3. Las conexiones alcanzan: dos réplicas con 10 conexiones cada una suman 20, muy por debajo del máximo de 100 de PostgreSQL.
4. No se propone agrandar el conjunto de conexiones, porque el experimento de la Tabla 7 mostró que no eleva el límite.

Como complemento de bajo costo, reducir el muestreo de trazas del 100 % al 10 % disminuiría el procesador que la propia instrumentación consume en cada petición. Las métricas de rendimiento no se perderían, porque no dependen del muestreo de trazas.

Limitaciones

La medición se hizo en un solo equipo, con todos los contenedores compartiendo el mismo procesador físico, aunque con límites de recursos asignados. La duración de tres minutos por escalón permite ver la saturación, pero no efectos de largo plazo como el crecimiento de la tabla o la acumulación de memoria. El desglose de trazas se basa en una muestra de 150 trazas por escalón, y el valor exacto del p95 cerca del umbral varió entre corridas (490 a 575 ms con 1200 usuarios), aunque la conclusión sobre el punto de saturación fue la misma en las tres.

Conclusiones

El entorno completo (aplicación, base de datos, APM y generador de carga) funciona en contenedores Docker y se levanta con un único comando. El APM permitió observar las peticiones por segundo, la latencia en percentiles y la tasa de error por operación, y desglosar cada petición entre la aplicación, la espera por conexión y la consulta a la base de datos.

Con los criterios declarados antes de medir, el límite de esta infraestructura es de 900 usuarios concurrentes y unas 895 peticiones por segundo, con 10 conexiones simultáneas a la base de datos. El recurso que impone ese límite es el procesador del contenedor de la aplicación, no la base de datos, la memoria ni el conjunto de conexiones. La lección más útil del ejercicio es que la métrica más llamativa del APM (la espera por conexiones, de hasta 99 % del tiempo) era un síntoma, y solo un experimento controlado permitió distinguir el síntoma de la causa.

Referencias

Anthropic. (2026). *Claude Code* (versión Claude Opus 5.5) [Herramienta de programación con modelo de lenguaje]. https://claude.com/claude-code

Carlson, B. (s. f.). *pg.Pool*. node-postgres. Recuperado el 19 de octubre de 2026, de https://node-postgres.com/apis/pool

Docker. (s. f.). *Docker Compose overview*. Docker Docs. Recuperado el 19 de octubre de 2026, de https://docs.docker.com/compose/

Grafana Labs. (s. f.-a). *docker-otel-lgtm* [Software]. GitHub. Recuperado el 19 de octubre de 2026, de https://github.com/grafana/docker-otel-lgtm

Grafana Labs. (s. f.-b). *Grafana k6 documentation*. Recuperado el 19 de octubre de 2026, de https://grafana.com/docs/k6/latest/

Gregg, B. (2020). *Systems performance: Enterprise and the cloud* (2.ª ed.). Addison-Wesley.

Little, J. D. C. (1961). A proof for the queuing formula: L = λW. *Operations Research*, *9*(3), 383–387. https://doi.org/10.1287/opre.9.3.383

OpenJS Foundation. (s. f.). *Don't block the event loop (or the worker pool)*. Node.js. Recuperado el 19 de octubre de 2026, de https://nodejs.org/en/learn/asynchronous-work/dont-block-the-event-loop

OpenTelemetry Authors. (s. f.). *JavaScript: Getting started*. OpenTelemetry. Recuperado el 19 de octubre de 2026, de https://opentelemetry.io/docs/languages/js/

Polania Cortes, J. C. (2026). *Taller de laboratorio N.º 4: Monitoreo con APM y prueba de estrés sobre el CRUD* [Guía de laboratorio]. Fundación Escuela Tecnológica de Neiva “Jesús Oviedo Pérez”.

PostgreSQL Global Development Group. (s. f.). *PostgreSQL 16 documentation*. Recuperado el 19 de octubre de 2026, de https://www.postgresql.org/docs/16/

Apéndice A

Instructivo para levantar el entorno desde cero

En un equipo que solo tenga Docker instalado, se abre una terminal en la carpeta del proyecto y se ejecutan los siguientes comandos:

1. Levantar la aplicación, la base de datos y el APM con un único comando: `docker compose up -d --build`.
2. Verificar la API en http://localhost:8080/health y el APM en http://localhost:3001 (panel "Laboratorio 4 → Lab 4 - CRUD productos (APM)").
3. Ejecutar la prueba de estrés con muestreo de recursos: `powershell -ExecutionPolicy Bypass -File scripts\medir.ps1 -Etiqueta corrida1`. En otros sistemas operativos, solo la carga: `docker compose --profile carga run --rm k6`.
4. Generar la tabla y las gráficas: `docker compose --profile analisis run --rm --build analisis corrida1`.
5. Apagar el entorno: `docker compose down` (conserva los datos) o `docker compose down -v` (borra también la base de datos).

El archivo README.md del repositorio amplía estas instrucciones y describe la estructura de carpetas, los endpoints del CRUD y la colección de Postman incluida.

Apéndice B

Entregables y su ubicación en el repositorio

Tabla 9

Entregables del laboratorio

Entregable	Ubicación
Código fuente de la aplicación instrumentada	app/src/server.js y app/src/tracing.js
Escenario de carga	k6/escenario.js
Definición de imágenes y composición	app/Dockerfile, analisis/Dockerfile y docker-compose.yml
Instructivo	README.md y Apéndice A
Declaración de criterios con fecha	docs/CRITERIOS_SATURACION.md y Tabla 2
Tabla de resultados	Tablas 4 a 6; resultados/corrida1/tabla.md
Gráficas	Figuras 1 a 6; resultados/corrida1/graficas/
Capturas del APM	Figuras 7 a 10; docs/capturas/
Ficha del equipo	Tabla 3 y docs/FICHA_EQUIPO.md
Respuestas de análisis	Sección Preguntas de análisis
Declaración de uso de IA	Apéndice C y docs/DECLARACION_USO_IA.md

Apéndice C

Declaración de uso de herramientas de inteligencia artificial

En este laboratorio se usó Claude Code, con el modelo Claude Opus 5.5 (Anthropic, 2026), un asistente de programación que se ejecuta en la terminal, el 19 de octubre de 2026. La herramienta se usó para construir la aplicación CRUD, sus imágenes de Docker y el archivo de composición; para configurar la instrumentación con OpenTelemetry, el APM y el panel de Grafana; para escribir el escenario de k6 y los scripts de muestreo y análisis; para proponer los criterios de saturación a partir de los valores sugeridos por la guía; y para redactar el borrador del instructivo y de este informe.

Los datos de las tablas y las figuras no fueron generados por la herramienta: provienen de la ejecución real de la prueba en el equipo descrito en la Tabla 3, consultados al APM y a las estadísticas de los contenedores por los scripts incluidos en el repositorio. Los datos sin procesar están en la carpeta resultados/. Los estudiantes revisaron el código y el informe, tomaron las capturas del APM y son responsables de sustentar cada conclusión.